NAME

codata - libray for fundamental physical constants

LIBRARY

codata (**-libcodata**, **-lcodata**)

SYNOPSIS

```
use codata
include "codata.h"
import pycodata
```

DESCRIPTION

codata is a Fortran library providing the fundamental physical constants according to CODATA (https://www.nist.gov/programs-projects/*codata*-values-fundamental-physical-constants). A C API allows usage from C, or can be used as a basis for other wrappers. A python wrapper allows easy usage from Python.

The latest *codata* constants (2022) https://pml.nist.gov/cuu/Constants were integrated in stdlib https://github.com/fortran-lang/stdlib/releases/tag/ since version 0.7.0. The constants are implemented as derived type which carries the name, the value, the uncertainty and the unit. This library is complementary to the constants defined in the stdlib by providing older values for the constants. The latest values (2022) do not have the year as a suffix in their name whereas older values (2010, 2014, 2018) feature the year as a suffix in their name.

All *codata* (physical) constants are defined as a derived type codata_constant_type. All the *codata* constants are provided as double precision reals. The names are quite long and can be aliased with shorter names. The derived type codata_constant_type defines 4 members and 2 procedures.

```
type :: codata_constant_type
    character(len=64) :: name
    real(dp) :: value
    real(dp) :: uncertainty
    character(len=32) :: unit
contains
    procedure :: print
    procedure :: to_real_sp
    procedure :: to_real_dp
    generic :: to_real => to_real_sp, to_real_dp
end type codata_constant_type
```

A module level interface to_real is available for getting the constant value or uncertainty of a constant.

o to_real(self, mold, uncertainty) result(r)

Get the constant value or uncertainty. Warning: Some constants cannot be converted to single precision sp reals due to the value of the exponents.

o class(codata_constant_type), intent(in) :: self

Codata constant

o real(sp), intent(in) :: mold

Should be of type real single or double precision

o logical, intent(in), optional :: uncertainty

Set to true if the uncertainty is required. Default to .false..

The C API exposes a structure codata_constant_type that defines the same members as in Fortran. to_real is not exposed in the C API.

```
type, bind(C) :: capi_constant_type
    character(kind=c_char) :: name(65)
    real(c_double) :: value
```

```
    real(c_double) :: uncertainty
    character(kind=c_char) :: unit(33)
end type capi_constant_type

typedef struct codata_constant_type{
    char name[65];
    double value;
    double uncertainty;
    char unit[33];
}cct;
```

The Python wrapper encapsulates the members in a dictionnary with the keys being name, value, uncertainty and unit. to_real is not exposed in the Python wrapper.

NOTES

To **use** *codata* within your **fpm**(1) (https://github.com/fortran-lang/fpm) project, add the following lines to your file:

```
[dependencies]
codata = { git="https://github.com/MilanSkocic/codata.git" }
```

EXAMPLE

Example in Fortran

```
program example_in_f
use codata
implicit none
print '(A)', '########## EXAMPLE IN FORTRAN ##########'
print '(A)', '# VERSION'
print *, "version = ", version()
print '(A)', '# CONSTANTS'
print *, "c = ",  SPEED_OF_LIGHT_IN_VACUUM%value
print '(A)', '# UNCERTAINTY'
print *, "u(c) = ", SPEED_OF_LIGHT_IN_VACUUM%uncertainty
print '(A)', '# OLDER VALUES'
print '(A, F23.16)', "Mu_2022(latest) = ", MOLAR_MASS_CONSTANT%value
print '(A, F23.16)', "Mu_2018 = ", MOLAR_MASS_CONSTANT_2018%value
print '(A, F23.16)', "Mu_2014 = ",  MOLAR_MASS_CONSTANT_2014%value
print '(A, F23.16)', "Mu_2010 = ",  MOLAR_MASS_CONSTANT_2010%value
end program
```

Example in C

```
#include <stdio.h>
#include "codata.h"
int main(void){
printf("########## EXAMPLE IN C ##########0);
printf("%s0,"# VERSION");
printf("version = %s0, codata_version());
printf("%s0,"# CONSTANTS");
printf("c = %f0, SPEED_OF_LIGHT_IN_VACUUM.value);
printf("%s0,"# UNCERTAINTY");
printf("u(c) = %f0, SPEED_OF_LIGHT_IN_VACUUM.uncertainty);
printf("%s0,"# OLDER VALUES");
printf("Mu_2022(latest) = %23.16f0, MOLAR_MASS_CONSTANT.value);
```

```
printf("Mu_2018 = %23.16f0, MOLAR_MASS_CONSTANT_2018.value);
printf("Mu_2014 = %23.16f0, MOLAR_MASS_CONSTANT_2014.value);
printf("Mu_2010 = %23.16f0, MOLAR_MASS_CONSTANT_2010.value);
return 0;
}
```

Example in Python

```
import sys
sys.path.insert(0, "../py/src/")
import pycodata
print("########## EXAMPLE IN PYTHON ##########")
print("# VERSION")
print(f"version = {pycodata.__version__}")
print("# Constants")
print("c =", pycodata.SPEED_OF_LIGHT_IN_VACUUM["value"])
print("# UNCERTAINTY")
print("u(c) = ", pycodata.SPEED_OF_LIGHT_IN_VACUUM["uncertainty"])
print("# OLDER VALUES")
print("Mu_2022 = ", pycodata.MOLAR_MASS_CONSTANT["value"])
print("Mu_2018 = ", pycodata.MOLAR_MASS_CONSTANT_2018["value"])
print("Mu_2014 = ", pycodata.MOLAR_MASS_CONSTANT_2014["value"])
print("Mu_2010 = ", pycodata.MOLAR_MASS_CONSTANT_2010["value"])
```

SEE ALSO

gsl(3), **codata**(1), **fpm**(1)